Quartermaster — Project Documentation

Oct 26, 2026 · @theot slebew

1. Executive summary

Quartermaster is an AI operations agent for community tech events that runs all of an event's money on PayPal: sponsor money in, purchases and people payouts out, under spending rules no agent can break.

One-liner: The treasurer every volunteer-run meetup, hackathon and student conference wishes it had, built on PayPal.

Problem. Community events run on volunteer time and sponsor money. Organizers pay out of pocket and chase reimbursements, sponsor payments arrive late, swag shows up after the event, and speakers abroad wait weeks to be paid. Sponsors rarely see where their money went, so they sponsor less next time.

Solution. Quartermaster connects to the event's own PayPal business account and runs the money lifecycle:

- **Money in:** the AI drafts and sends sponsor invoices; tickets and donations come in through PayPal checkout.
- **Rules:** organizers write spending rules in plain English; the AI translates them into fixed rules enforced in code. Budget unlocks only when sponsor money has actually cleared.
- **Plan:** a goal like "200-person hackathon on Mar 14" becomes a Gantt plan with lead times and dependencies.
- **Buy:** the agent sources swag, prizes and supplies through Channel3 and pays automatically within the rules, or asks for PayPal approval above them.
- **Pay people:** speaker reimbursements, volunteer receipt claims and prize money go out through PayPal Payouts, to PayPal or Venmo.
- **Protect:** late deliveries trigger a refund and a re-sourced order, and the Gantt shows whether event day is at risk.
- **Report:** each sponsor gets an AI-written report of exactly where their money went.
- **Kill switch:** one click revokes the agent's PayPal token and all spending stops.

Why it wins.

- PayPal is the whole system, not a checkout button: about ten PayPal products each do a job the event cannot run without.
- Every sponsor tool is load-bearing: AG Studio (command center), Bryntum (event plan), Channel3 (sourcing), Render Workflows (money timers), APIMatic (how we built the integration).
- Most of the judging panel are developer advocates and community managers who run events themselves.

Pitch line: "Every hackathon organizer has paid for pizza out of their own pocket. Quartermaster makes sure no one has to again."

2. Hackathon context

The official deadline is Nov 12, 2026 at 12:00 PM PST, which is 03:00 WIB on Nov 13; our internal submission date is Nov 10 because MAJU:ON runs Nov 11–12. Source: Devpost page.

Fact	Detail
Event	PayPal AI Hackathon: Build what's next with PayPal and AI
Official deadline	Nov 12, 2026, 12:00 PM PST (Nov 13, 03:00 WIB)
Our deadline	Nov 10, 2026 (submit, then edit until the official deadline if needed)
Registered participants	21,169 as of Oct 26, 2026
Core requirement	Meaningful use of both PayPal (sandbox) and AI
Team	4 people, about 20 hours each per week (about 160 person-hours total)
Budget	Sponsor credits first; at most $20 of our own money

Judging criteria and how we score

Criterion	What judges ask	How Quartermaster scores
Technological Implementation	Thorough, skilled, non-trivial use of PayPal and AI	About ten PayPal products in one working loop; agents with real tools and code-enforced rules
Design	A complete, coherent product, not a proof of concept	One polished app: plan, dashboard, approvals and reports share one design system
Potential Impact	A credible, specific problem for a real audience	Volunteer organizers who front costs; sponsors who want proof of spend
Innovation / Idea	Creative, novel, different from existing concepts	Budget that unlocks only on cleared sponsor funds; agent spending authority with a PayPal kill switch
Presentation	Video shows it working end to end and explains who it is for	Story-driven 3-minute video built backward from the demo script (section 10)

Prizes we target

Prize	Amount	Our angle	Priority
Best Use of PayPal + AI	$5,000	PayPal is the event's whole money system, driven by agents	Primary
Best Use of Agentic Commerce	$5,000	Procurement agent buys within rules, automatic or approved	Primary
Best Use of AG Grid	$5,000 / $2,000 / 3 × $1,000	AG Studio command center with custom widgets and a custom agent	Primary

Prize	Amount	Our angle	Priority
Best Use of Bryntum	3 × $1,000	Event plan where a late PayPal payment visibly blocks the next phase	Primary
Best Use of Channel3	$1,500	All product sourcing and price comparison	Primary
Best Use of APIMatic	3 × $1,000 + 6 months Business	PayPal integration built with the Context Plugin, documented	Primary
Best Use of Render	$1,000 / $750 / $500 credits	Hosting plus Render Workflows for money timers	Secondary
Best Demo Delivery	$5,000	Strong story video	Secondary
Most Impactful	$5,000	Volunteer organizers worldwide	Secondary
1st / 2nd / 3rd overall	$12,000 / $8,000 / $5,000	Falls out of doing the above well	Secondary

The panel includes six PayPal staff plus sponsor leaders such as Sylwia Vargas (AG Grid), Mats Bryntse (CEO, Bryntum), Ignacio Valdez Bicard (Channel3), Ameer Hassan (APIMatic) and Shifra Williams (Render).

3. Problem and users

Our users are the volunteers who run community tech events: meetups, hackathons and student conferences funded by sponsors and run on hard deadlines.

Personas

Persona	Who they are	Main pain today	What Quartermaster gives them
Lead organizer	Runs the event, sets the plan and budget	Juggles vendors, deadlines and money in spreadsheets and chats	One plan, one budget, an agent doing the legwork

Persona	Who they are	Main pain today	What Quartermaster gives them
Treasurer	Volunteer responsible for the money	Fronts costs, reconciles receipts, chases sponsors	Rules the agent must follow, approvals only when needed, automatic books
Volunteer	Buys small things on the day	Pays out of pocket and waits weeks for reimbursement	Snap a receipt, get paid to PayPal or Venmo within the rules
Speaker	Often travels from another country	Slow, awkward cross-border reimbursement	Payout straight to their PayPal account
Sponsor	Company funding the event	Pays an invoice, never sees how it was spent	Invoice by PayPal and an itemized spend report afterwards
Prize winner	Participant who won	Prize money arrives late or needs bank details	Batch payout to PayPal or Venmo after judging

Pain points we solve

1. **Money in arrives late or not at all.** Sponsor invoices are manual and reminders are awkward.
2. **Organizers front costs.** Volunteers pay personally because the event's money isn't accessible or approved in time.
3. **Deadlines are hard.** Swag, prizes and equipment must arrive before event day; a late delivery is discovered too late.
4. **Paying people is slow.** Speakers, volunteers and winners in different countries wait for payment.
5. **No accountability.** Sponsors get no clear report, and there is no audit trail for who spent what and why.

Why this niche

- **Hard deadline:** event day cannot move, so the Gantt plan is essential, not decorative.
- **Money flows both ways:** money in (sponsors, tickets) and money out (purchases, people), all on PayPal.

- **Global:** meetups and student clubs exist everywhere, and cross-border payouts are what PayPal does well.
- **Judge resonance:** most judges are developer advocates and community managers who run events.
- **Founder-market fit:** as hackathon participants and student organizers, we have seen these problems first-hand.

4. Product overview

One event moves through seven stages, and every stage runs on PayPal; the demo event below is the running example for the whole document.

Demo event: *OpenHack 2027* — a 200-person hackathon on Mar 14, 2027, with an $8,000 budget funded by three sponsors and ticket sales.

The event lifecycle

1. **Set up.** The treasurer signs in with PayPal and connects the event's PayPal business account (the *treasury*). The organizer enters date, headcount, budget and categories.
2. **Money in.** The AI drafts sponsor invoices from the sponsorship tiers and sends them through PayPal Invoicing. Tickets and donations come in through PayPal checkout. Each payment updates the unlocked budget live.
3. **Rules.** The treasurer writes rules in plain English: "Nothing over $200 without my approval. Swag only from the approved list. Volunteers can claim up to $50 per day." The AI turns them into structured rules shown back for confirmation.
4. **Plan.** The organizer states the goal. The planning agent builds a Gantt plan: venue deposit, swag (with its lead time), prizes, AV, catering, speaker travel. Purchases that need sponsor money depend on that sponsor's invoice being paid.
5. **Buy.** The procurement agent sources items through Channel3, compares options and builds carts. Carts within the rules are paid automatically with the saved PayPal token; carts above the threshold go to the treasurer as a PayPal approval.
6. **Pay people.** Volunteers upload receipts; the AI extracts and checks them against the rules and pays approved claims. Speaker reimbursements and prize money go out as Payouts batches.
7. **Protect and report.** Shipments are tracked; a late item triggers a refund and a replacement order, and the Gantt shows the effect on event day. After the event, each sponsor receives a report of where their money went.

At any time, the treasurer can hit the **kill switch**: the saved PayPal token is revoked and the agent can no longer spend.

What the user sees

Screen	Purpose	Built with
Event home	Countdown, unlocked budget, alerts, chat with the agent	React
Plan	Gantt of tasks, purchases and payment dependencies	Bryntum Gantt
Money	Sponsor funding, spend vs budget, payouts ledger, rules status	AG Studio
Approvals	Carts and claims waiting for the treasurer, each with the agent's reasoning	React + PayPal JS SDK
Rules	Plain-English rules and their structured version	React
Claims	Volunteers upload receipts and see status	React (mobile-friendly)
Sponsor report	Shareable per-sponsor spend report	React

5. Feature specification

25 features in three tiers: Core must work end to end by Nov 3, Upgrades land Nov 4–7, Stretch only if we are ahead. Owners map to the roles in section 11.

ID	Feature	What it does	Done when	Priority	Owner
F1	Sign in with PayPal	Treasurer and organizers sign in with Log in with PayPal	Sign-in returns verification status and PayPal account ID; user saved	Core	P2
F2	Treasury connection	Links the event's PayPal business account	Balance and recent transactions show in the app	Core	P2
F3	Event setup	Date, headcount, budget, categories	Event saved; countdown and budget visible	Core	P4

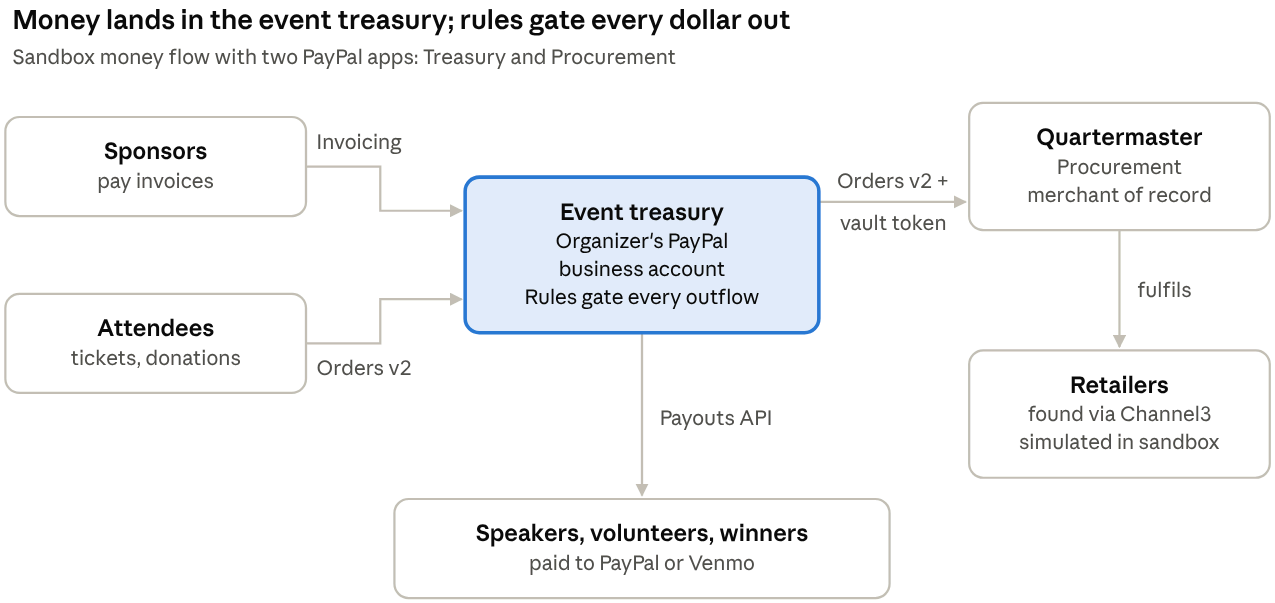
ID	Feature	What it does	Done when	Priority	Owner
F4	Sponsor invoices	AI drafts an invoice from the sponsor's tier and sends it through PayPal Invoicing	Invoice arrives in the sponsor's sandbox account; paid invoice raises the unlocked budget	Core	P3
F5	Invoice reminders	Agent sends a friendly reminder for overdue invoices	Reminder sent through the API and logged	Upgrade	P3
F6	Tickets and donations	PayPal checkout on the public event page	Captured payment raises the unlocked budget	Upgrade	P2
F7	Monthly community partners	Recurring sponsorship for monthly meetups	Subscription activates; each payment adds budget	Stretch	P3
F8	Rules compiler	Plain-English rules become structured rules the treasurer confirms	The 5 demo rules compile correctly; edits recompile	Core	P1
F9	Rules engine	Code checks every spend before money moves	Off-rule spends are blocked or routed to approval; every check is logged	Core	P1
F10	Funds-gated budget	Spending allowed only up to cleared funds minus commitments	An unpaid sponsor invoice blocks the purchases that depend on it	Core	P1
F11	Planning agent	Goal becomes a plan with tasks, lead times and dependencies	Demo goal yields at least 15 linked tasks rendered in Bryntum	Core	P1 + P4
F12	Sourcing	Channel3 search, compares options, picks with reasoning	Cart holds real products with prices and links	Core	P1
F13	Automatic payment	Saved PayPal token pays carts within the rules	Order captured with no treasurer action	Core	P2
F14	Supervised approval	Carts above the threshold open a PayPal approval	Treasurer approves in PayPal; order captured	Core	P2
F15	Kill switch	Revokes the saved PayPal token	Token deleted; the next agent spend is refused; webhook logged	Core	P2
F16	Receipt claims	Receipt photo → AI extraction → rules check → payout	Approved claim paid to a sandbox PayPal or Venmo recipient	Upgrade	P3

ID	Feature	What it does	Done when	Priority	Owner
F17	Speaker and prize payouts	Batch payouts to many recipients and countries	Batch succeeds; payouts ledger updates	Upgrade	P3
F18	Shipment tracking	Tracking added to captured orders; delivery dates feed the plan	Tracking shows on the order; Gantt dates update	Upgrade	P4
F19	Late delivery recovery	Late item triggers refund, re-sourcing and a plan update	Refund issued, new order placed, dependent tasks shift	Upgrade	P4
F20	Command center	AG Studio dashboard with custom, themed widgets	4 custom widgets work with cross-filtering	Core	P3
F21	Dashboard agent	Custom agent works with AG Studio's built-in agents	"Where are we overspending?" builds a new widget	Upgrade	P3
F22	Sponsor report	AI-written per-sponsor report from PayPal records	Shareable page ties each spend to sponsor funds	Upgrade	P3
F23	Audit log	Every agent action with its reason and rule result	Visible on the Approvals and Money screens	Core	P1
F24	MCP server	Exposes Quartermaster tools to Claude or ChatGPT	Claude Desktop can request a purchase within the rules	Stretch	P1
F25	PayPal disputes	Opens and tracks disputes when a vendor fails	Confirmed working in the day-1 test	Stretch	P4

6. PayPal integration map

Quartermaster uses 13 PayPal capabilities across two sandbox apps: all money in lands in the event treasury, and the rules engine gates every dollar out.

Money architecture



money flow · 6 parties, 2 PayPal apps

Sponsors and attendees pay the treasury; the treasury pays Quartermaster Procurement for goods (automatically by saved token, or after approval) and pays people directly through Payouts.

- **Treasury app:** the event's own PayPal business account. For the hackathon, the event connects a sandbox business account's REST credentials. In production, events would be onboarded through PayPal Multiparty.
- **Procurement app:** Quartermaster's merchant account, acting as merchant of record for physical goods. It saves the treasury's PayPal wallet as a token, so the agent never touches card data.
- **Fulfilment:** buying from the real retailer is simulated in sandbox; Channel3 buy links show the actual source.
- **Agent Ready and Store Sync** need onboarding through PayPal's request form. We submit the form on day 1, ask on Discord about sandbox access, and build on public APIs either way.

PayPal capabilities used

#	Capability	Endpoint(s)	App	Used for	Features
1	Log in with PayPal (Identity)	`/v1/oauth2/token`, `/v1/identity/oauth2/userinfo`	Treasury	Sign-in; account verification status	F1
2	Invoicing v2	`POST /v2/invoicing/invoices`, `/send`, `/remind`	Treasury	Sponsor invoices and reminders	F4, F5
3	Orders v2 (income)	`POST /v2/checkout/orders`, `/capture`	Treasury	Tickets and donations	F6
4	Subscriptions	`/v1/billing/plans`, `/v1/billing/subscriptions`	Treasury	Monthly community partners	F7
5	Vault v3 payment tokens	`/v3/vault/setup-tokens`, `/v3/vault/payment-tokens`, `DELETE /v3/vault/payment-tokens/{id}`	Procurement	Agent wallet and kill switch	F13, F15
6	Orders v2 (spend)	`POST /v2/checkout/orders` with `vault_id` or approval link	Procurement	Automatic and supervised purchases	F13, F14
7	Package tracking	`POST /v2/checkout/orders/{id}/track`	Procurement	Delivery dates for the plan	F18
8	Refunds	`POST /v2/payments/captures/{id}/refund`	Procurement	Late delivery recovery	F19
9	Disputes	`/v1/customer/disputes`	Procurement	Vendor failure cases	F25
10	Payouts	`POST /v1/payments/payouts`	Treasury	Claims, speakers, prize winners	F16, F17
11	Transaction Search	`GET /v1/reporting/transactions`	Treasury	Reconciliation and sponsor reports	F20, F22
12	Webhooks + simulator	`/v1/notifications/webhooks`	Both	Live updates and demo triggers	All

#	Capability	Endpoint(s)	App	Used for	Features
13	Agent Toolkit / MCP server	`@paypal/agent-toolkit` (sandbox mode)	Both	Agent tool calls for invoices, orders, tracking	F4, F12, F18

Transaction Search is not real-time, so live numbers come from webhooks and Transaction Search is used for reconciliation and reports.

Webhooks and what they trigger

Webhook event	Quartermaster action
`INVOICING.INVOICE.PAID`	Unlock the sponsor's budget share; release dependent Gantt tasks
`PAYMENT.CAPTURE.COMPLETED`	Record income or spend; update the dashboard
`CHECKOUT.ORDER.APPROVED`	Capture the approved cart
`VAULT.PAYMENT-TOKEN.CREATED`	Mark the agent wallet active
`VAULT.PAYMENT-TOKEN.DELETED`	Confirm the kill switch; disable agent spending
`PAYMENT.PAYOUTSBATCH.SUCCESS`	Mark claims, speaker payments or prizes as paid
`PAYMENT.CAPTURE.REFUNDED`	Close the late-delivery case
`BILLING.SUBSCRIPTION.ACTIVATED`	Add the recurring partner to the budget
`CUSTOMER.DISPUTE.CREATED`	Open a case in the audit log

7. AI architecture

Five agents share one tool layer, and no agent can move money directly: every money action passes through the rules engine, which is plain code, not a prompt.

Agents

Agent	Job	Tools it can call	Human gate
Rules compiler	Turns plain-English rules into structured JSON rules	`propose_rules` (write-only draft)	Treasurer confirms before rules go live
Planning agent	Turns the event goal into tasks, lead times and dependencies	`create_tasks`, `link_task_to_invoice`, `read_event`	Organizer can edit the plan freely
Procurement agent	Sources products, compares options, builds carts, pays	Channel3 search, `build_cart`, `request_payment` (goes to the rules engine)	Approval above the threshold or outside the rules
Treasury agent	Drafts sponsor invoices and reminders, reads receipts, prepares payouts	Invoicing tools, receipt extraction (vision), `request_payout` (goes to the rules engine)	Treasurer approves payout batches above the threshold
Dashboard agent	Answers questions and builds widgets in AG Studio	AG Studio built-in agents, Transaction Search reader	None (read-only)

Guardrails

1. **Rules are code.** The model only drafts rules. Enforcement is a deterministic function: `check(spend, rules, budget) → allow | needs_approval | block`, with the reason logged.
2. **No tool, no action.** Agents only get the tools listed above. The PayPal capture and payout calls are reachable only through the rules engine.
3. **Funds-gated budget.** Unlocked budget = cleared income − committed spend. An agent cannot commit money that has not arrived.
4. **Kill switch.** Deleting the payment token removes the agent's ability to pay, enforced by PayPal, not by our code.
5. **Untrusted input is data.** Product pages, receipts and invoice notes are treated as data, never as instructions.

6. **Full audit log.** Every agent action stores the input, the reasoning summary, the rule result and the PayPal ID.

Models and cost

- **Runtime:** Vercel AI SDK, so the model can be swapped by configuration. Develop on a free-tier model; record the demo with the strongest model we can afford, within the $20 cap.
- **Claude Pro** covers Claude Code for building (and the APIMatic Context Plugin). It does not include API credits for the running app.
- **Structured output:** every agent returns zod-validated JSON; a failed parse retries once, then falls back to asking the human.
- **Vision:** receipt extraction uses a vision-capable model with a fixed schema (merchant, date, total, currency, line items).
- **Hosted demo protection:** per-session rate limit and a seeded demo event, so judges cannot exhaust our quota.

8. Sponsor integrations

We use five sponsor tools, each doing a job the product needs; Elastic, KERNEL, Zapier, Astropods and Postman are skipped because they have no prize and add risk.

Sponsor	What we build	What its judges asked for	How we show it
AG Studio	Money command center: sponsor funding, spend vs budget by category, payouts ledger, rules status, delivery risk	Polish (custom widgets, theming, layout) and the Studio Agent Framework: a custom agent talking to Studio's built-in agents	4 custom widgets in our brand theme; dashboard agent builds a widget live in the video
Bryntum	Event plan Gantt: tasks, lead times, dependencies; sponsor invoices as gating tasks	Their own example: a late PayPal payment visibly blocks the next phase; an agent that drives the schedule	Paid invoice releases tasks; late delivery cascades toward event day

Sponsor	What we build	What its judges asked for	How we show it
Channel3	All product sourcing: search, compare, product detail, buy links	Real API calls as a meaningful part of the product	Agent compares 3 options per item and explains its pick
Render	Hosting (web, API, Postgres) and Render Workflows for durable money jobs	Deploy on Render; use Workflows to coordinate agents and background jobs	Workflow runs for invoice reminders, delivery checks and payout batches, shown in the README
APIMatic	PayPal integration built with the Context Plugin in Claude Code	Best use of the Context Plugin	README section with prompts and screenshots; claim the free subscription

AG Studio widgets

Widget	Shows	Type
Funding thermometer	Pledged vs invoiced vs cleared, per sponsor	Custom widget
Budget burn	Spend vs unlocked budget by category, days to event	Custom widget
Payouts ledger	Claims, speaker payments and prizes with status	Grid
Rules health	Spends allowed, sent to approval, blocked, with reasons	Custom widget
Delivery risk	Items whose expected arrival is after their deadline	Custom widget

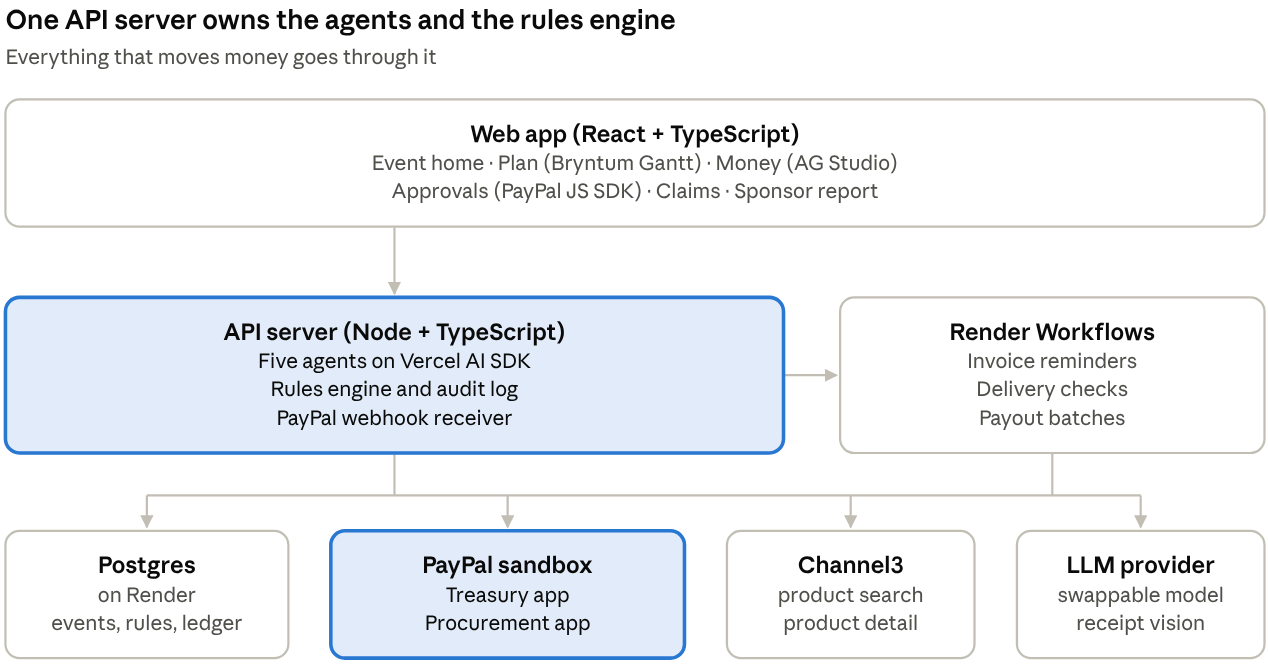
Bryntum plan rules

- Each sponsor invoice is a task; purchases funded by that sponsor depend on it.
- Expected delivery dates come from PayPal tracking data and push dependent tasks.
- Event day is a fixed milestone; any task pushed past its deadline turns red and alerts the organizer.

- The planning agent edits the plan through the same API a human uses, so people can adjust anything by drag and drop.

9. System architecture

A React web app talks to one Node API server that holds the agents, the rules engine and the webhook receiver; Render Workflows runs the long-lived money jobs, all deployed on Render.



system components · web, API, workflows, 4 services

The web app never calls PayPal's server APIs directly; PayPal's webhooks come back into the API server, which updates Postgres and the plan.

Stack

Layer	Choice
Frontend	React + TypeScript (Vite), AG Studio (React), Bryntum Gantt (React wrapper), PayPal JS SDK for approval buttons
Backend	Node + TypeScript (Hono or Express), Vercel AI SDK, `@paypal/agent-toolkit`, Channel3 SDK, zod

Layer	Choice
Data	Postgres on Render, Drizzle ORM
Jobs	Render Workflows (TypeScript SDK)
Hosting	Render web service for the app and API; public demo URL
Repo	One public GitHub monorepo, MIT license, `.env.example`, seed script for the demo event

Data model

Table	Key fields
`events`	id, name, date, headcount, budget_total, treasury_merchant_id
`sponsors`	id, event_id, name, tier, pledged, paypal_invoice_id, status
`rules`	id, event_id, source_text, compiled_json, version, confirmed_by
`tasks`	id, event_id, name, start, end, depends_on[], deadline, funded_by_sponsor_id
`carts`	id, task_id, items (Channel3 IDs, prices), total, decision, reason
`orders`	id, cart_id, paypal_order_id, capture_id, tracking_number, expected_delivery, status
`claims`	id, person, receipt_url, extracted_json, amount, decision, payout_item_id
`payouts`	id, paypal_batch_id, items, status
`ledger`	id, direction (in/out), amount, currency, source, paypal_id, sponsor_id, created_at
`agent_wallet`	id, vault_token_id, status, revoked_at

Table	Key fields
`audit_log`	id, actor, action, input, reason, rule_result, paypal_id, created_at

Workflows

Workflow	Trigger	Steps
Budget recompute	Any money webhook	Recompute unlocked budget; unblock or block dependent tasks
Invoice reminders	Daily (manual trigger for the demo)	Find overdue invoices; draft a friendly reminder; send; log
Delivery check	Hourly (manual trigger for the demo)	Compare expected delivery with the task deadline; if late, refund, re-source and shift the plan
Payout batch	Claims approved, or prizes confirmed	Group items; run rules; create the PayPal payout batch; wait for the webhook
Sponsor report	After the event	Pull Transaction Search; attribute spend to sponsors; AI writes the report; publish the page

10. Demo video script

The video runs 2:50 and is the main thing judges see, so we lock this script on Oct 28 and build backward from it.

Time	Beat	On screen	Proves
0:00–0:15	Hook	Organizer paying for pizza on a personal card; voice: "Every hackathon organizer has paid for pizza out of their own pocket."	Problem, audience

Time	Beat	On screen	Proves
0:15–0:30	Set up	Sign in with PayPal; connect the treasury; OpenHack 2027, 200 people, Mar 14, $8,000	Identity, treasury
0:30–0:50	Money in	Agent drafts 3 sponsor invoices; switch to a sponsor's sandbox account and pay one; the unlocked budget rises live	Invoicing, webhooks, funds-gated budget
0:50–1:05	Rules	Treasurer types 3 rules; structured rules appear for confirmation	Rules compiler
1:05–1:30	Plan	Goal typed; the Gantt builds itself; swag waits on a sponsor's payment, then unlocks	Bryntum, planning agent
1:30–1:55	Buy	Agent compares Channel3 options; a $140 cart pays automatically; a $420 prize cart opens a PayPal approval	Agentic commerce, vault, approval
1:55–2:15	Late swag	Delivery check flags late lanyards; refund, re-sourced order, Gantt cascades and event day stays green	Tracking, refunds, Render Workflows
2:15–2:35	Pay people and report	Volunteer receipt paid to Venmo; prize payouts to winners in 3 countries; dashboard agent builds a "spend by sponsor" widget	Payouts, AG Studio agent
2:35–2:50	Kill switch and close	Treasurer revokes the token; the agent tries to buy and is refused; voice: "Every guarantee in Quartermaster is a PayPal guarantee."	Trust, PayPal as the system

Production notes

- Record in the sandbox with seeded data; trigger webhooks with PayPal's webhook simulator or our demo controls.

- Screen capture at 1080p, voice-over recorded separately, captions burned in.
- No third-party logos or copyrighted music; use generic product categories (lanyards, adapters, snacks) so brands don't dominate the screen.
- Upload to YouTube as public; keep the video under 3:00.

11. Team roles

Four roles, each owning one prize target; PayPal work is split across three people so no one becomes a bottleneck. Assign names at the kickoff.

Role	Owns	Features	Prize focus	Name
P1 Agent brain	Agent loop, rules compiler, rules engine, funds-gated budget, Channel3 sourcing, planning agent logic, audit log	F8–F12, F23, F24	Agentic Commerce, Channel3	
P2 PayPal buy side and infra	Log in with PayPal, treasury connection, vault, automatic and supervised payment, kill switch, tickets, Render deploy and Workflows, APIMatic write-up	F1, F2, F6, F13–F15	PayPal + AI, Render, APIMatic	
P3 PayPal money in/out and AG Studio	Sponsor invoices and reminders, subscriptions, receipt claims, payouts, Transaction Search, command center, dashboard agent, sponsor report	F4, F5, F7, F16, F17, F20–F22	AG Grid, Most Impactful	

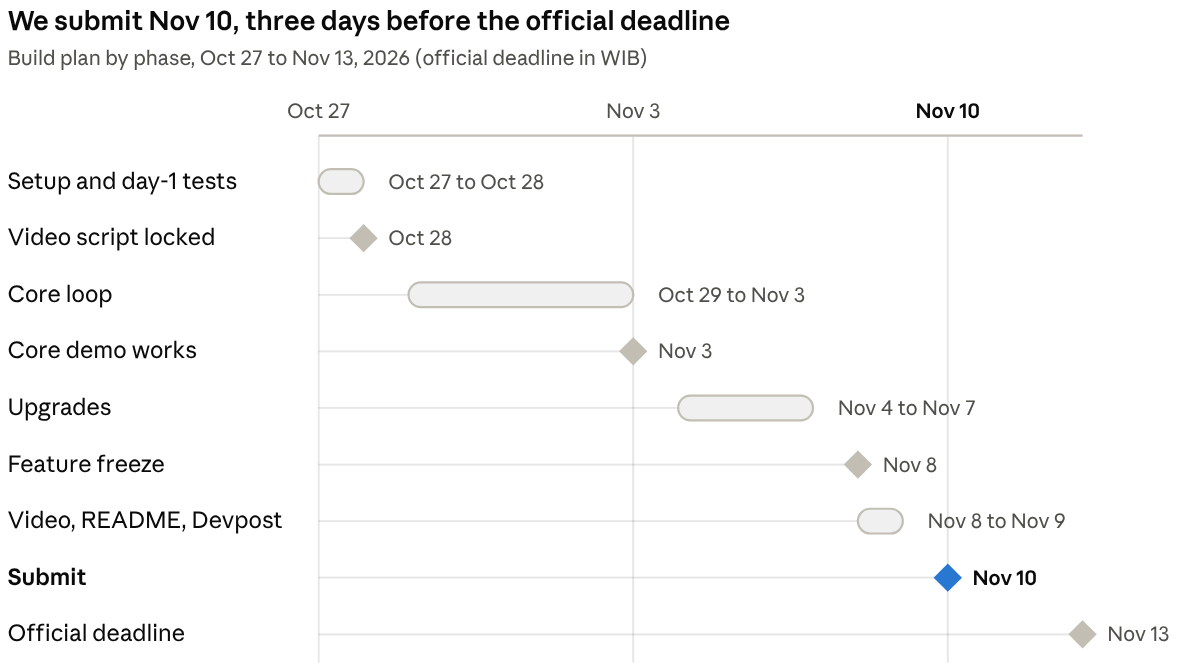
Role	Owns	Features	Prize focus	Name
P4 Plan, protection and demo	Event setup, Bryntum Gantt, tracking, late-delivery recovery, disputes, UI design system, video, Devpost write-up	F3, F11 (UI), F18, F19, F25	Bryntum, Best Demo Delivery	

Working agreements

- **Daily 15-minute standup** at a fixed time; blockers first.
- **One shared contract:** API types live in a shared package; change them by pull request only.
- **Main always deploys:** every merge goes to the Render demo URL.
- **Each owner keeps their sponsor's README section current** with screenshots as features land.
- **Everyone records** short screen clips of finished features for the video.

12. Timeline

We have 15 days and about 160 person-hours; the core loop must work end to end by Nov 3 so the last week goes to upgrades and the video.



build timeline · 6 phases, 3 gates, Oct 27 to Nov 13

Each phase ends at a gate; if a gate slips, we cut from the Upgrades tier (section 13) instead of moving the submit date.

Gates

- [ ] **Oct 28 — Setup done:** all accounts and trials active; every day-1 test in section 14 answered; demo script locked; repo and Render deploy live.
- [ ] **Nov 3 — Core demo works:** sign in, invoice paid, rules confirmed, plan generated, one automatic and one approved purchase, dashboard updates, kill switch blocks spending.
- [ ] **Nov 7 — Upgrades done:** late-delivery recovery, payouts, dashboard agent and sponsor report working on the hosted URL.
- [ ] **Nov 8 — Feature freeze:** only bug fixes after this point.
- [ ] **Nov 9 — Assets ready:** video uploaded to YouTube; README with setup steps and sponsor sections; Devpost text drafted.
- [ ] **Nov 10 — Submitted** on Devpost; hosted demo checked from a clean browser.

Daily plan for setup week

Day	P1	P2	P3	P4
Oct 27	Channel3 key and search test; agent skeleton	PayPal developer account, two sandbox apps, vault test	Invoicing and Payouts tests; AG Studio trial and boilerplate	Bryntum trial; Gantt hello world; repo and design tokens
Oct 28	Rules schema and engine stub	Render deploy; webhooks listener; APIMatic plugin set up	Transaction Search test; first widget	Demo script draft; app shell and navigation

13. Scope tiers and cut list

The Core tier alone is a complete, winnable demo; everything else is added only after the Nov 3 gate passes.

Tier	Includes	Done by	If it slips
Core	Sign in, treasury, sponsor invoices, rules compiler and engine, funds-gated budget, planning agent and Gantt, Channel3 sourcing, automatic and approved payment, kill switch, command center, audit log	Nov 3	Stop everything else and fix Core
Upgrades	Invoice reminders, tickets, receipt claims, speaker and prize payouts, tracking, late-delivery recovery, dashboard agent, sponsor report	Nov 7	Cut in the order below
Stretch	Monthly partners (Subscriptions), MCP server, PayPal disputes	Only if ahead	Mention as roadmap in the README

Cut order if we fall behind

1. Stretch features (all of them).
2. Invoice reminders and tickets.

3. Sponsor report (keep a static mock in the README instead).
4. Dashboard agent (keep the custom widgets).
5. Receipt claims (keep prize payouts, which are one Payouts batch).

Never cut

- The PayPal approval popup next to an automatic payment.
- The Gantt task that unlocks when a sponsor's invoice is paid.
- The kill switch.
- The rules engine as code.

14. Day-1 tests, risks and mitigations

Nine checks on Oct 27–28 decide the architecture; each has a fallback, so no single result can sink the project.

Day-1 tests

- [ ] **Vault (P2):** save a sandbox PayPal wallet as a payment token, then charge it with no buyer present. Fallback: supervised approval for every cart.
- [ ] **Business account as payer (P2):** confirm the treasury business account can approve and pay a Procurement order. Fallback: use a sandbox personal account as the treasury's payer wallet.
- [ ] **Invoicing (P3):** create, send and pay an invoice between sandbox accounts; confirm `INVOICING.INVOICE.PAID` arrives. Fallback: Orders checkout link for sponsors.
- [ ] **Payouts (P3):** send a batch to a PayPal and a Venmo sandbox recipient; confirm webhooks. Fallback: PayPal-only payouts.
- [ ] **Tracking and refunds (P4):** add tracking to a captured order and refund a capture. Fallback: store tracking in our database only.
- [ ] **Disputes (P4):** check whether a sandbox dispute can be opened and read through the API. Fallback: drop to Stretch, use refunds.
- [ ] **Channel3 coverage (P1):** search lanyards, adapters, bulk snacks, keyboards, T-shirts; confirm prices and buy links come back.
- [ ] **Trials (P3, P4):** AG Studio and Bryntum trial licenses active; ask on Discord whether keys may sit in a public repo and stay valid through judging.

- [ ] **Access request (P2):** submit PayPal's agentic commerce services form and ask on Discord about hackathon sandbox access.

Risks

Risk	Likelihood	Impact	Mitigation
Vaulting not enabled for our sandbox app	Medium	Loses automatic mode	Test day 1; fall back to approval for all carts
Trial licenses expire during judging (45 days from Oct 27 is about Dec 11)	Medium	Hosted demo breaks	Confirm judging dates; ask sponsors on Discord; keep video as proof
LLM quota exhausted by judges	Medium	Demo fails for judges	Rate limit, seeded demo event, cached plans for the demo goal
Scope too big for 160 hours	High	Unfinished features	Gates and cut order (section 13); Core first
AG Studio and Bryntum learning curve	Medium	Slow UI work	One dedicated owner each; start from the PayPal + AG Studio boilerplate
Agent makes a wrong purchase	Low	Bad demo moment	Rules engine as code; approval threshold; seeded, rehearsed flows
Retail purchase is simulated	Certain	Credibility question	Say it plainly in README and video: Procurement is merchant of record; fulfilment is out of scope
Trademarks in the video	Medium	Rule breach	Generic products, no logos, no copyrighted music
Render service sleeping or out of credits	Low	Demo URL slow or down	Paid instance on sponsor credits; check the URL daily during judging

15. Submission checklist and open questions

Everything below must be done by Nov 10; each item maps to a Devpost requirement or a sponsor prize.

Devpost requirements

- [ ] Project built with PayPal sandbox and AI, both central
- [ ] Text description of features and functionality
- [ ] Hosted demo URL that judges can use (seeded demo event, sandbox test accounts listed)
- [ ] Setup and run instructions in the README as a backup to the hosted URL
- [ ] Tools section: every model and AI service, and how each is used
- [ ] Public GitHub repo with all source and assets; MIT license visible in the About section
- [ ] Demo video under 3:00, public on YouTube, shows the product working; no third-party trademarks or copyrighted music

Sponsor prize evidence

- [ ] README section per sponsor: AG Studio, Bryntum, Channel3, Render, APIMatic (what we built, screenshots, where in the code)
- [ ] APIMatic: Context Plugin prompts and screenshots; free subscription form submitted
- [ ] Render: Workflows dashboard screenshot; services listed
- [ ] PayPal: list of capabilities used, linking to code paths

Open questions

- [ ] Who takes each role (P1–P4)?
- [ ] Which free-tier model do we develop on, and do we spend part of the $20 on a stronger model for the video recording?
- [ ] When does judging end, and will trial licenses and Render credits last until then?
- [ ] Can hackathon teams get sandbox access to Agent Ready or Store Sync?
- [ ] One credible, sourced figure on volunteer organizers or community events for the pitch's impact line.
- [ ] Final product name: keep Quartermaster or rename for the events niche?

16. Sources

Pages opened while preparing this document, as of Oct 26, 2026.

- PayPal AI Hackathon on Devpost — deadline, judging criteria, judges, prizes, participant count
- Introducing the AG Studio Agent Framework — harness, runners, agents, tools
- Hackathon: PayPal and AG Studio — AG Grid and Bryntum prizes, boilerplate, idea starters
- PayPal agentic commerce services overview — Agent Ready, Store Sync, access form

Reference docs to read during the build (from search results; verify details before relying on them):

- PayPal Agent Toolkit and GitHub repo
- PayPal Payouts API and Payouts sandbox testing
- PayPal save PayPal wallet with Payment Method Tokens
- PayPal WebMCP and Agent Ready blog
- Channel3 UCP and API docs
- Render Workflows announcement